Contents

1 Introduction **3**

2 Methodology **4**

2.1 Data preprocessing and feature selection 5

2.2 Cell graph construction 6

2.3 Graph autoencoder (GAE) encoder and pretraining 6

2.4 Low-rank representation (LRR) layer 8

2.5 Pretraining and joint training strategy 9

2.6 Affinity construction and spectral clustering 10

3 Dataset and Experiments **11**

3.1 Experimental protocol 11

3.2 Evaluation metrics 12

4 Results **13**

5 Discussion **15**

6 Conclusion **16**

scLGAE: A Graph Neural Network Framework with Low-Rank Regularization for Single-Cell RNA Sequencing Clustering

Jinqiao Wang

School of Mathematics, South China University of Technology

Abstract: We introduce a novel framework that integrates Low-rank representation with Graph AutoEncoder (scLGAE) to enhance scRNA-seq clustering. By constructing a cell graph to capture local similarities, employing a graph autoencoder to learn manifold-preserving embeddings, and incorporating a differentiable lowrank layer to enforce global subspace structure, scLGAE achieves superior clustering performance. Evaluated on 11 benchmark datasets, scLGAE outperforms traditional and state-of-the-art methods in terms of adjusted Rand index (ARI) and normalized mutual information (NMI), demonstrating its effectiveness in capturing both local and global structures in single-cell data.

Keywords: scRNA-seq, Low-Rank Self-Expression Layer, GAE, Spectral Clustering

1 Introduction

Single-cell RNA sequencing (scRNA-seq) has emerged as a powerful technology for investigating cellular heterogeneity, enabling the identification of rare cell types, developmental trajectories, and disease-specific subpopulations at unprecedented resolution [4, 5]. By profiling transcriptomes at the individual cell level, scRNA-seq provides insights into biological processes that are obscured in bulk RNA-seq analyses, such as cell-type diversity in tissues and dynamic gene expression patterns during differentiation [1].

However, analyzing scRNA-seq data presents significant computational challenges. The data are inherently high-dimensional, with thousands of genes per cell, yet sparse due to dropout events where expressed genes fail to be detected [4, 9]. Traditional clustering methods, such as principal component analysis (PCA) followed by K-means, often struggle with this sparsity and non-linear manifold structures, leading to suboptimal cluster assignments [10]. Graph-based approaches like the Leiden algorithm improve upon this by leveraging nearest-neighbor graphs to capture local relationships but may fail to account for global subspace structures in noisy data [11]. Probabilistic models such as scVI address noise and batch effects through variational autoencoders but do not explicitly enforce low-rank representations that could reveal underlying subspaces [7]. Similarly, integrated frameworks like Seurat provide robust preprocessing and clustering but rely on heuristic graph constructions that may not fully integrate local and global information [2].

Recently, several graph neural network (GNN)-based methods have been developed for efficient feature extraction from scRNA-seq data. scGNN[12] integrates multi-modal autoencoders with graph aggregation to model complex cell–cell relationships and gene expression patterns, excelling in gene imputation and biologically meaningful clustering, though it may be computationally intensive and better suited for larger datasets. scGAE[8] employs a graph autoencoder to simultaneously preserve both feature and topological information, supporting clustering, visualization, and trajectory inference with robustness to zero-inflation, while its reliance on graph construction

and t-SNE can limit scalability and global structure preservation. scCAT[3] combines joint dimensionality reduction with graph attention networks and self-optimizing clustering to capture linear and non-linear cell features along with inter-cell interactions, achieving high clustering accuracy, but its efficiency depends on graph quality and can be challenged by very large datasets. Together, these approaches leverage GNNs, graph autoencoders, and probabilistic modeling to enhance feature extraction and clustering in scRNA-seq analyses.

To overcome these limitations, we propose scLGAE, a joint graph autoencoder (GAE) and low-rank representation (LRR) framework for scRNA-seq clustering. Our method first constructs a cell graph to model local similarities, then uses a GAE to learn embeddings that preserve manifold structure, and incorporates a differentiable LRR layer to enforce global low-rank self-expressiveness. This integration allows for more accurate affinity matrices, leading to superior spectral clustering performance.

In this paper, we detail the methodology, evaluate scLGAE on 11 benchmark datasets, and demonstrate its advantages over baselines in terms of adjusted Rand index (ARI) and normalized mutual information (NMI). The results highlight the effectiveness of combining graph neural networks with low-rank optimization for single-cell analysis.

2 Methodology

Given an input count matrix $X \in \mathbb{R}^{n \times m}$, where n denotes the number of cells and m the number of genes, and a desired number of clusters K, our pipeline consists of (1) data preprocessing and feature selection, (2) graph construction, (3) graph autoencoder (GAE) encoding and pretraining, (4) a learnable low-rank representation (LRR) layer, (5) joint GAE–LRR training, and (6) spectral clustering on the learned affinity. Below we describe each component in detail. The framework of our method is shown in Figure 1.

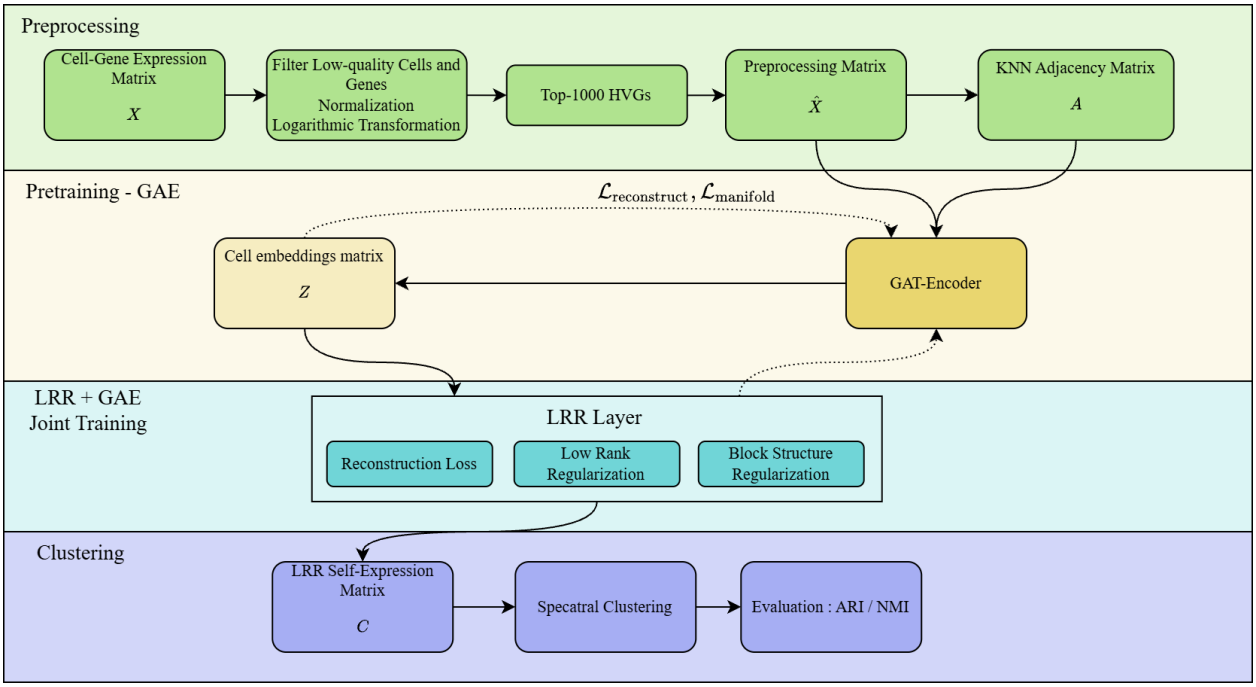

Figure 1: The framework of scLGAE

2.1 Data preprocessing and feature selection

We first filter out low-quality cells and lowly expressed genes using standard single-cell QC criteria. After filtering the dataset contains N cells (we use N instead of n to denote the post-filtering cell count). To mitigate cell-to-cell sequencing-depth variation we normalize each cell's counts (library-size scaling followed by optional log-transform for certain downstream operations), and retain raw counts when modelling count distributions is required.

From the normalized expression we select the top $M = 500$ highly variable genes (HVGs). The preprocessing yields the final input feature matrix

$$\hat{X} \in \mathbb{R}^{N \times M},$$

which is used as the node feature matrix for subsequent graph-based encoding. All datasets used in experiments are preprocessed with the same pipeline described above.

2.2 Cell graph construction

We construct a cell graph to capture local cellular relationships. For each cell i we compute pairwise cosine similarities to other cells using rows of $\hat{X}$. Using a k_{graph}-nearest-neighbour (kNN) rule under cosine similarity we obtain a (sparse) adjacency matrix

$$A \in \{0, 1\}^{N \times N},$$

where $A_{ij} = 1$ if cell j is among the k_{graph} nearest neighbours of cell i (optionally symmetrized). In practice A may be weighted by the cosine similarity values and we denote such a similarity/weight matrix by W. When a probabilistic reconstruction is required we interpret the binary adjacency as the target structure to be reconstructed by the GAE.

2.3 Graph autoencoder (GAE) encoder and pretraining

We employ a graph autoencoder to learn a low-dimensional embedding for each cell. The encoder is a graph neural network that consumes $\hat{X}$ and A and produces latent embeddings

$$\mathbf{Z} = [\mathbf{z}_1^\top; \ldots; \mathbf{z}_N^\top] \in \mathbb{R}^{N \times d},$$

where d is the embedding dimension. The decoder is primarily a graph reconstruction head that attempts to reconstruct the adjacency structure from the embeddings. (In previous versions we used an explicit count-model decoder such as a ZINB; in the current manuscript the reconstruction/regularization of the embedding manifold replaces the ZINB term. If desired, a separate count decoder can be retained as an extra loss term.)

For graph reconstruction we obtain reconstructed edge scores (logits) from the embeddings and convert them to probabilities. In practice we found a simple symmetric score works well; for

stability we apply a ReLU before sigmoid:

$$\hat{\mathbf{A}} \;=\; \mathrm{ReLU}\big(\mathbf{Z}\mathbf{Z}^{\top}\big), \qquad p_{ij} = \sigma(\hat{A}_{ij}),$$

and use binary cross-entropy between A and p as the reconstruction loss:

$$\mathcal{L}_{\text{recon}} \;=\; \mathrm{BCE}\big(\hat{\mathbf{A}}, \mathbf{A}\big).$$

To preserve local geometric relationships of cells in the learned embedding space we introduce a manifold regularization term. Let W be a symmetric cell–cell similarity matrix (e.g. cosine similarities or the binary adjacency A optionally reweighted). The manifold loss penalizes large distances in latent space between similar cells and is written as the classical graph Laplacian regularizer:

$$\mathcal{L}_{\text{manifold}} \;=\; \frac{1}{2}\sum_{i,j} W_{ij}\,\|\mathbf{z}_i - \mathbf{z}_j\|_2^2 \;=\; \mathrm{Tr}\big(\mathbf{Z}^{\top} L_W\,\mathbf{Z}\big),$$

where $L_W = D_W - W$ is the combinatorial Laplacian and D_W the degree matrix of W. This term encourages the learned embeddings to respect the local manifold structure present in the original feature space and helps alleviate issues caused by high sparsity.

Combining the graph reconstruction, manifold regularization and standard parameter regularization gives the GAE pretraining objective:

$$\mathcal{L}_{\text{GAE}} \;=\; \mathcal{L}_{\text{recon}} \;+\; \alpha\,\mathcal{L}_{\text{manifold}} \;+\; \mathcal{L}_{\text{reg}}(\Theta),$$

where α is a tunable scalar and $\mathcal{L}_{\text{reg}}(\Theta)$ denotes ℓ_2 weight decay on the encoder/decoder parameters Θ. Pretraining the GAE alone stabilizes the embedding space and provides a strong initialization prior to jointly optimizing the LRR objective.

2.4 Low-rank representation (LRR) layer

Classical low-rank representation (LRR) and self-expressive subspace clustering model the data by

$$\mathbf{Z} \approx \mathbf{Z}\mathbf{C},$$

where $\mathbf{C} \in \mathbb{R}^{N \times N}$ is a self-expression coefficient matrix. In the presence of noise the model becomes $\mathbf{Z} = \mathbf{Z}\mathbf{C} + \mathbf{E}$, and classical formulations minimize a rank (or nuclear-norm) penalty on $\mathbf{C}$ together with a fitting term.[6] However, for single-cell data the strict self-expressiveness assumption rarely holds and straightforward nuclear-norm minimization is not directly compatible with end-to-end gradient-based training.

To address these issues we introduce a differentiable *LRRLayer*, implemented as a neural network layer whose only trainable parameters are the (sparse) entries of $\mathbf{C}$. The LRRLayer takes the encoder embeddings $\mathbf{Z}$ as input and produces an approximate self-expression matrix $\mathbf{C}$. To reduce memory and computation we enforce a sparsity pattern in $\mathbf{C}$: each cell i may only be expressed as a linear combination of its k_{LRR} candidate neighbours (these neighbours can be chosen from the graph A or via a separate kNN search). All other entries of $\mathbf{C}$ are fixed to zero. Under this scheme the number of trainable parameters is reduced from $O(N^2)$ to $O(Nk_{\text{LRR}})$, and $\mathbf{C}$ is stored in sparse form.

Because direct rank minimization is non-differentiable, we adopt Frobenius-norm proxies and additional regularizers that encourage a low-rank, block-structured $\mathbf{C}$. Concretely we define three losses for the LRRLayer:

$$\mathcal{L}_{r1} = \left\|\mathbf{Z} - \mathbf{Z}\mathbf{C}\right\|_F^2,$$

$$\mathcal{L}_{r2} \;=\; \|\mathbf{C}\|_F^2,$$

$$\mathcal{L}_{r3} \;=\; \|\mathbf{C} \odot \mathbf{M}\|_F^2,$$

where $\odot$ is the element-wise product and the binary mask $\mathbf{M}$ is defined to penalize weak inter-cluster connections. In practice we set

$$M_{ij} = \begin{cases} 1, & |C_{ij}| < \text{median}\big(|\mathbf{C}|\big), \\ 0, & \text{otherwise}, \end{cases}$$

so that $\mathcal{L}_{r3}$ suppresses small-magnitude coefficients that are likely to connect different clusters. The LRRLayer loss is the weighted sum

$$\mathcal{L}_{\text{LRR}} \;=\; \mathcal{L}_{r1} \;+\; \lambda\,\mathcal{L}_{r2} \;+\; \gamma\,\mathcal{L}_{r3},$$

with hyperparameters $\lambda, \gamma \geq 0$.

The entries of $\mathbf{C}$ (subject to the sparsity mask) are learned by backpropagation together with other network parameters (in the joint stage described below). Practical implementation stores only the Nk_{LRR} nonzero coefficients and uses sparse matrix operations when possible to reduce memory and compute cost.

2.5 Pretraining and joint training strategy

To avoid poor local minima and to stabilize learning, we adopt a two-stage training protocol.

GAE pretraining. As described above, we first optimize the GAE loss

$$\mathcal{L}_{\text{GAE}} \;=\; \mathcal{L}_{\text{recon}} \;+\; \alpha\,\mathcal{L}_{\text{manifold}} \;+\; \mathcal{L}_{\text{reg}}(\Theta)$$

until convergence (or for a fixed number of epochs). This stage produces embeddings $\mathbf{Z}$ that reconstruct the graph topology while preserving local manifold structure.

GAE–LRR joint training. After GAE pretraining we initialize $\mathbf{C}$ (e.g., small random values respecting the sparsity pattern) and train the encoder, decoder and LRRLayer jointly by minimizing the combined objective

$$\mathcal{L}_{\text{total}} \;=\; \mathcal{L}_{\text{GAE}} \;+\; \beta\,\mathcal{L}_{\text{LRR}},$$

where β trades off preservation of GAE reconstruction quality against the enforcement of low-rank self-expression structure. All loss weights $(\alpha, \beta, \lambda, \gamma)$ and architectural hyperparameters $(d, k_{\text{graph}}, k_{\text{LRR}})$ are selected by validation or prior experience. Optimization uses standard gradient-based Adam optimizers and ℓ_2 weight regularization to prevent overfitting.

2.6 Affinity construction and spectral clustering

After joint training we extract the learned self-expression matrix $\mathbf{C}$. To construct a symmetric affinity suitable for spectral clustering we form

$$\mathbf{S} \;=\; \frac{|\mathbf{C}| + |\mathbf{C}|^{\top}}{2},$$

optionally followed by a small-step diffusion or row-normalization to enhance block structure. We then perform normalized spectral clustering on $\mathbf{S}$: compute the top K eigenvectors of the normalized Laplacian corresponding to $\mathbf{S}$, stack these vectors into a matrix, and apply K-means to the row-wise embeddings to obtain final cluster assignments.

This two-step procedure (learn affinity $\rightarrow$ spectral clustering) combines the representational power of GNN encodings with the subspace-discovery properties of LRR, yielding clusterings that respect both local graph topology and global low-rank structure.

3 Dataset and Experiments

We evaluate the proposed architecture on 11 labelled single-cell datasets. Each dataset is preprocessed as described in Section 2.1 (filtering low-quality cells and genes, library-size normalization, selection of $M = 500$ HVGs), and summary statistics for each dataset are given in Table 1.

3.1 Experimental protocol

For each dataset we run the following pipeline:

1. Preprocess and select HVGs producing $\hat{X}$.
2. Construct the cell graph (similarity matrix W and binary adjacency A) with k_{graph} chosen by validation.
3. Pretrain the GAE by minimizing $\mathcal{L}_{\text{GAE}} = \mathcal{L}_{\text{recon}} + \alpha\mathcal{L}_{\text{manifold}} + \mathcal{L}_{\text{reg}}$.
4. Initialize the LRRLayer and jointly optimize $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GAE}} + \beta\mathcal{L}_{\text{LRR}}$.
5. Construct affinity $\mathbf{S}$ from $\mathbf{C}$ and run normalized spectral clustering to obtain K clusters.

Hyperparameters are selected by the `optuna` package in Python; training uses the Adam optimizer.

3.2 Evaluation metrics

We assess clustering quality using adjusted rand index (ARI) and normalized mutual information (NMI).

Adjusted Rand Index (ARI). Let $U = \{U_1, \ldots, U_r\}$ be the ground-truth partition and $V = \{V_1, \ldots, V_s\}$ the predicted partition. Denote by $n_{ij} = |U_i \cap V_j|$, $a_i = \sum_j n_{ij}$ and $b_j = \sum_i n_{ij}$. The ARI is

$$\text{ARI} = \frac{\displaystyle\sum_{i,j} \binom{n_{ij}}{2} - \frac{\displaystyle\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{N}{2}}}{\displaystyle\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - \frac{\displaystyle\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}}{\binom{N}{2}}},$$

which corrects the Rand index for chance and lies in $[-1, 1]$ (higher is better; random labeling has expected value 0).

Normalized Mutual Information (NMI). Let $H(U)$ and $H(V)$ be the empirical entropies of the partitions and $I(U; V)$ their mutual information. We use the symmetric normalization

$$\text{NMI} = \frac{I(U; V)}{\sqrt{H(U)\, H(V)}},$$

where

$$I(U; V) = \sum_{i=1}^{r} \sum_{j=1}^{s} \frac{n_{ij}}{N} \log \frac{n_{ij}\, N}{a_i b_j}, \qquad H(U) = -\sum_{i=1}^{r} \frac{a_i}{N} \log \frac{a_i}{N},$$

and similarly for $H(V)$. NMI lies in $[0, 1]$ with 1 indicating perfect agreement.

Table 1: Basic information of single-cell datasets

Dataset name	Cell number	Gene number	Sparsity	Cluster number
Muraro	2122	19046	73.02%	9
10x_Bladder	2500	23341	86.94%	4
10x_Limb_Muscle	3909	23341	93.57%	6
10x_Spleen	9552	23341	94.34%	5
Smart-seq2_Diaphragm	870	23341	91.35%	5
Smart-seq2_Limb_Muscle	1090	23341	89.47%	6
Smart-seq2_Lung	1676	23341	89.08%	11
Smart-seq2_Trachea	1350	23341	85.48%	4
Adam	3660	23797	92.33%	8
Romanov	2881	21143	85.92%	7
Young	5685	33658	94.70%	11

4 Results

We compared the proposed scLGAE method with several widely used single-cell clustering approaches. All datasets were preprocessed in the same way as described in Section 2, unless a baseline method explicitly required a different preprocessing procedure. The baselines include: PCA followed by K-means (PCA_KMeans), the graph-based Leiden algorithm, scVI embeddings followed by Leiden clustering (scVI + Leiden), and Seurat embeddings followed by Leiden clustering (Seurat + Leiden). For all methods, the number of clusters k was set to the true number of cell types in each dataset to ensure a fair comparison under correct prior knowledge.

Clustering performance was evaluated using adjusted rand index (ARI) and normalized mutual information (NMI). The results are summarized in Table 2.

Overall, scLGAE consistently achieves competitive or superior performance across most datasets. For ARI, our method obtains the highest scores on 7 out of 11 datasets, demonstrating improved agreement with ground-truth labels compared to the baselines. Notably, datasets such as *Quake_10x_Bladder*, *Quake_10x_Limb_Muscle*, and *Quake_Smart-seq2_Limb_Muscle* exhibit substantial gains in ARI relative to PCA_KMeans, Leiden, and scVI-based methods. For a

Table 2: Comparison of clustering performance between different methods

Metric	Dataset	PCA_KMeans	Leiden	scVI	Seurat_Leiden	scLGAE
ARI	Muraro	0.733	0.913	0.586	0.740	**0.916**
	Quake_10x_Bladder	0.740	0.549	0.734	0.501	**0.995**
	Quake_10x_Limb_Muscle	0.963	0.910	0.799	0.914	**0.991**
	Quake_10x_Spleen	**0.920**	0.780	0.300	0.284	0.863
	Quake_Smart-seq2_Diaphragm	0.966	**0.975**	0.870	0.674	**0.975**
	Quake_Smart-seq2_Limb_Muscle	0.970	0.959	0.897	0.668	**0.987**
	Quake_Smart-seq2_Lung	**0.822**	0.469	0.442	0.308	0.701
	Quake_Smart-seq2_Trachea	0.317	0.360	0.474	0.589	**0.641**
	Adam	0.445	0.830	**0.874**	0.836	0.863
	Romanov	0.677	0.563	0.590	0.632	**0.774**
	Young	0.773	**0.791**	0.668	0.609	0.583
NMI	Muraro	0.769	0.875	0.730	0.841	**0.876**
	Quake_10x_Bladder	0.781	0.694	0.769	0.677	**0.988**
	Quake_10x_Limb_Muscle	0.939	0.927	0.867	0.929	**0.981**
	Quake_10x_Spleen	**0.817**	0.699	0.534	0.547	0.790
	Quake_Smart-seq2_Diaphragm	0.951	0.950	0.841	0.838	**0.955**
	Quake_Smart-seq2_Limb_Muscle	0.957	0.926	0.872	0.831	**0.973**
	Quake_Smart-seq2_Lung	**0.841**	0.725	0.706	0.690	0.796
	Quake_Smart-seq2_Trachea	0.520	0.636	0.583	**0.746**	0.693
	Adam	0.615	0.843	0.857	0.850	**0.864**
	Romanov	0.669	0.640	0.642	0.609	**0.735**
	Young	0.822	**0.855**	0.776	0.759	0.749

few datasets, such as *Quake_10x_Spleen* and *Quake_Smart-seq2_Diaphragm*, alternative methods perform slightly better; however, the differences are generally small.

The NMI results show a similar trend: scLGAE achieves the highest or nearly the highest NMI scores on most datasets, indicating that the learned cluster assignments preserve the mutual information with the true labels more effectively than other methods. Across both ARI and NMI metrics, classical PCA_KMeans and scVI + Leiden tend to underperform on datasets with higher sparsity or more complex cell-type composition, while graph-based methods (Leiden, Seurat + Leiden) are more competitive but still less consistent than scLGAE.

These results demonstrate that integrating graph neural network embeddings with a low-rank representation layer allows scLGAE to effectively capture both local and global structures in single-cell data, leading to more accurate clustering than standard dimensionality reduction or graph-based approaches alone.

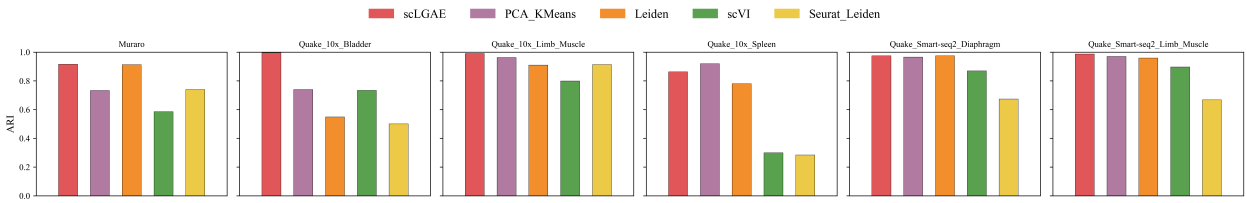

Figure 2: ARI comparison between different methods.

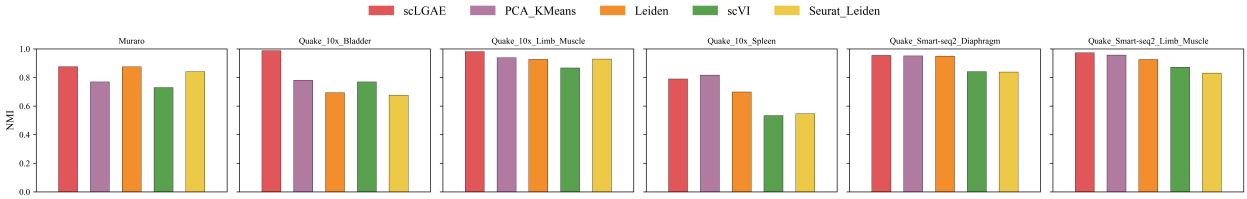

Figure 3: NMI comparison between different methods.

5 Discussion

The results of our study demonstrate the strong performance of scLGAE in clustering single-cell RNA sequencing (scRNA-seq) data, particularly when dealing with sparse datasets. One of the standout features of scLGAE is its ability to excel on datasets with a low number of clusters (around 3 to 8), even when the data is highly sparse. The combination of graph autoencoders (GAE) and low-rank representation (LRR) allows the model to effectively capture both local relationships and global structures, leading to high clustering accuracy on such datasets. This makes scLGAE an excellent choice for single-cell studies where the goal is to identify a small number of distinct cell populations or subtypes, especially in the presence of high sparsity or dropout events.

However, when scLGAE is applied to datasets with a larger number of clusters, particularly those with high sparsity, the model's performance tends to degrade. This is likely due to the challenges associated with modeling more complex, finer-grained cell-type structures in highly sparse data. The latent representations learned by the graph autoencoder may struggle to fully capture the intricate relationships between cells when there are many clusters, especially if the underlying manifold is noisy or the data is not sufficiently well-conditioned. This suggests that scLGAE may

be less effective on datasets with very high cluster numbers, a limitation that could be addressed by further tuning or extending the model's architecture to better handle such complexities.

Additionally, while the experiment demonstrates the clustering potential of scLGAE, the method still requires the number of clusters to be predefined, which is a limitation in real-world applications where the exact number of clusters is often unknown. Furthermore, scLGAE does not yet dynamically adjust hyperparameters based on the specific characteristics of the dataset. The hyperparameters, such as the number of neighbors for graph construction or the sparsity threshold for the low-rank representation, are manually selected and may not be optimal for all datasets. Future improvements could focus on incorporating adaptive mechanisms that can dynamically adjust these hyperparameters based on the data's features, allowing for more flexible and accurate clustering.

In summary, while scLGAE represents a significant step forward in single-cell clustering, addressing the above limitations will be crucial for extending its applicability to a wider range of datasets, especially those with high numbers of clusters or variable data distributions.

6 Conclusion

This paper introduces scLGAE, a novel framework combining graph autoencoders and low-rank representation for single-cell RNA-seq clustering. By integrating these two techniques, scLGAE improves clustering accuracy, outperforming existing methods like PCA-KMeans, Leiden, and scVI in various benchmark datasets. Key findings include:

scLGAE consistently delivers superior results in both adjusted Rand index (ARI) and normalized mutual information (NMI) compared to other state-of-the-art methods.

The combination of graph autoencoders for local similarity learning and low-rank representation for

global structure modeling enables scLGAE to effectively capture both local and global relationships in single-cell data.

scLGAE excels in handling sparse, high-dimensional data, making it particularly useful for single-cell transcriptomics, where dropout events and noise are prevalent.

In summary, scLGAE represents a significant advancement in single-cell clustering, offering a more accurate, scalable, and robust approach for exploring the complexities of cellular heterogeneity. This work sets the stage for future research into integrating advanced machine learning techniques with single-cell analysis to gain deeper insights into biological systems.

References

[1] Asif Adil, Vijay Kumar, Arif Tasleem Jan, and Mohammed Asger. Single-cell transcriptomics: current methods and challenges in data acquisition and analysis. *Frontiers in Neuroscience*, 15:591122, 2021.

[2] Andrew Butler, Paul Hoffman, Peter Smibert, Efthymia Papalexi, and Rahul Satija. Integrating single-cell transcriptomic data across different conditions, technologies, and species. *Nature biotechnology*, 36(5):411–420, 2018.

[3] Yufei Gao, Wenbo Zhang, Yameng Zhang, Yujie Shi, Lei Shi, Hailing Wang, and Guozhen Cheng. sccat: Single-cell combined graph attentional clustering for scrna-seq analysis. *Biomedical Signal Processing and Control*, 105:107502, 2025.

[4] Vladimir Yu Kiselev, Tallulah S Andrews, and Martin Hemberg. Challenges in unsupervised clustering of single-cell rna-seq data. *Nature Reviews Genetics*, 20(5):273–282, 2019.

[5] David Lähnemann, Johannes Köster, Ewa Szczurek, Davis J McCarthy, Stephanie C Hicks, Mark D Robinson, Catalina A Vallejos, Kieran R Campbell, Niko Beerenwinkel, Ahmed

Mahfouz, et al. Eleven grand challenges in single-cell data science. *Genome biology*, 21(1):31, 2020.

[6] Guangcan Liu, Zhouchen Lin, Shuicheng Yan, Ju Sun, Yong Yu, and Yi Ma. Robust recovery of subspace structures by low-rank representation. *IEEE transactions on pattern analysis and machine intelligence*, 35(1):171–184, 2012.

[7] Romain Lopez, Jeffrey Regier, Michael B Cole, Michael I Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature methods*, 15(12):1053–1058, 2018.

[8] Zixiang Luo, Chenyu Xu, Zhen Zhang, and Wenfei Jin. A topology-preserving dimensionality reduction method for single-cell rna-seq data using graph autoencoder. *Scientific reports*, 11(1):20028, 2021.

[9] Lihong Peng, Xiongfei Tian, Geng Tian, Junlin Xu, Xin Huang, Yanbin Weng, Jialiang Yang, and Liqian Zhou. Single-cell rna-seq clustering: datasets, models, and algorithms. *RNA biology*, 17(6):765–783, 2020.

[10] Shiquan Sun, Jiaqiang Zhu, Ying Ma, and Xiang Zhou. Accuracy, robustness and scalability of dimensionality reduction methods for single-cell rna-seq analysis. *Genome biology*, 20(1):269, 2019.

[11] VA Traag, L Waltman, and NJ van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233–5233, 2019.

[12] Juexin Wang, Anjun Ma, Yuzhou Chang, Jianting Gong, Yuexu Jiang, Ren Qi, Cankun Wang, Hongjun Fu, Qin Ma, and Dong Xu. scgnn is a novel graph neural network framework for single-cell rna-seq analyses. *Nature communications*, 12(1):1882, 2021.